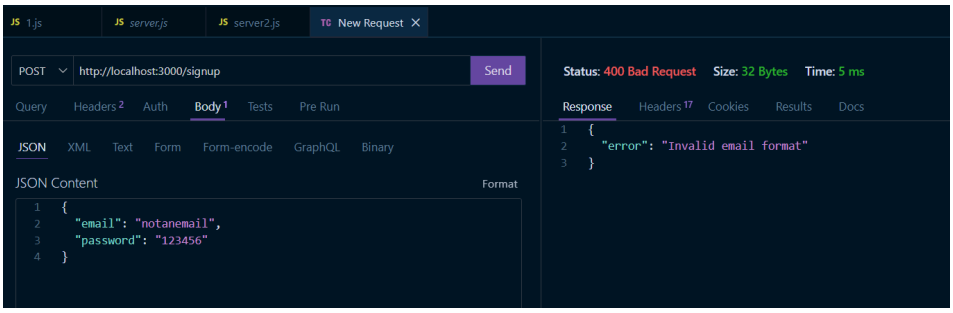
Week 3: Advanced Security and Final Reporting

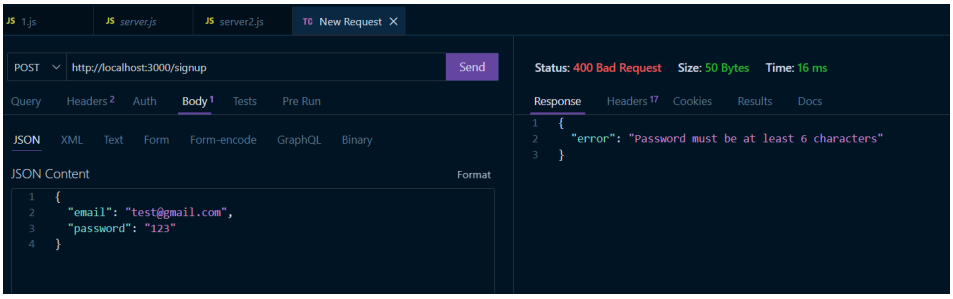
1. Basic Penetration Testing

In this week, basic penetration testing techniques were applied to test the security of the developed Node.js application.

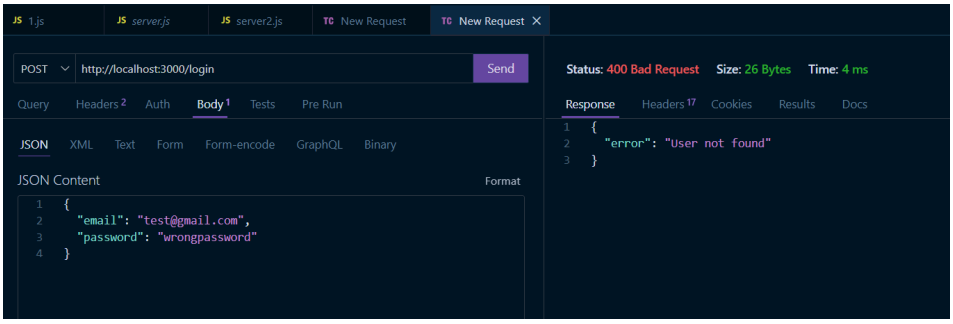
Activities Performed:

- Tested authentication endpoints (`/login`, `/profile`) using Thunder Client.
- Attempted SQL Injection by sending malicious input such as:
  {
  "email": "' OR 1=1 --",
  "password": "123456"
  }
- Verified that invalid or malicious inputs were properly rejected.



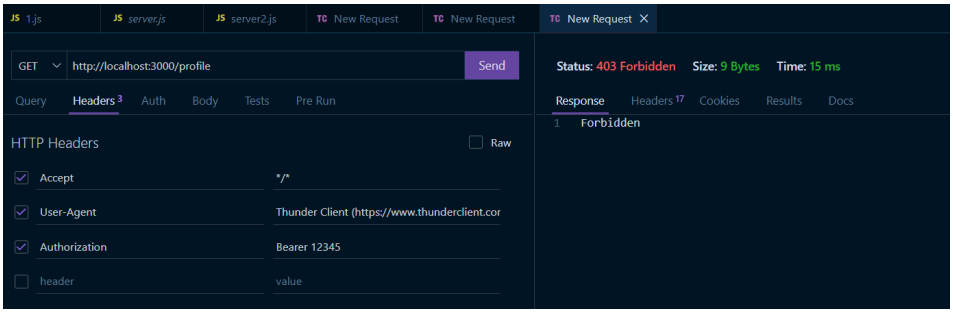


- Confirmed that protected routes could not be accessed without a valid JWT token.



Outcome:

- The application correctly blocked unauthorized access and returned 403 Forbidden responses.



- Input validation prevented injection attacks.
- Authentication middleware ensured only logged-in users could access protected routes.

2. Set Up Basic Logging

Basic logging was implemented to monitor HTTP requests and application activity.

Implementation:

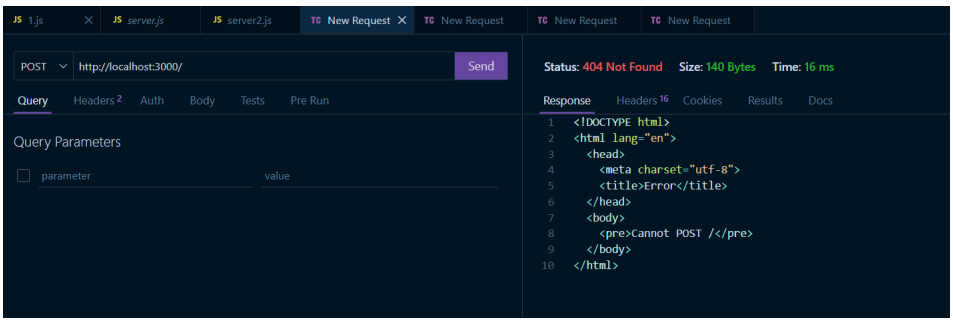
- Installed and configured the **Morgan** logging middleware.
- Enabled logging for:
    - HTTP request method
    - Endpoint URL
    - Response status codes
    - Request timestamps

Purpose:

- To track incoming requests and detect suspicious behavior.
- To help in debugging and monitoring application activity.
- To maintain visibility of failed login attempts and unauthorized access.

Outcome:

- All requests (GET, POST, failed attempts) were logged in the terminal.
- Logs provided clear insight into how users interact with the application.



3. Create a Simple Security Checklist

A simple security checklist was created to ensure basic security practices are consistently followed.

Security Checklist:

- ✅ Input validation and sanitization enabled
- ✅ Passwords hashed using bcrypt
- ✅ JWT-based authentication implemented
- ✅ Protected routes secured with middleware
- ✅ HTTP security headers configured using Helmet
- ✅ Request logging enabled using Morgan
- ✅ Proper error handling without information leakage
- ✅ Unauthorized access properly blocked

Purpose:

- To maintain security awareness during development.
- To reduce common vulnerabilities such as injection attacks and unauthorized access.
- To provide a reference for future improvements.